Building Trust in LLM-Based Code Translation through Transcompiler-Assisted Prompting

AMIRA T. MAHMOUD*, Center for Informatics Science (CIS), School of Information Technology and Computer Science, Nile University, Egypt

NAHLA ELARABY, Institute of Computer Technology, Technical University of Vienna (TU Wien), Austria and Canadian International College-CIC, Egypt

HALA ZAYED, Faculty of Engineering, Egypt University of Informatics, Egypt and Faculty of Computer and Artificial intelligence, Department of Information Systems, Benha University, Egypt

AHMED H YOUSEF, El Sewedy University of Technology, Egypt and Ain Shams University, Faculty of Engineering, Computers & Systems Department, Egypt

WALAA MEDHAT, Center for Informatics Science (CIS), School of Information Technology and Computer Science, Nile University, Egypt and Faculty of Computer and Artificial intelligence, Department of Information Systems, Benha University, Egypt

Recent advances in Large Language Models (LLMs) have led to notable progress in automated code generation and translation tasks, yet concerns about trustworthiness and semantic reliability remain a challenge. Previous approaches often rely on fine-tuning with large amounts of parallel training data, which is impractical for low-resource languages where data scarcity amplifies error rates and undermines user confidence. In this paper, we present a hybrid approach, TC-assisted prompting, that combines transcompiler-based translators and LLMs to enhance code translation and increase trust in their outputs. The approach integrates transcompiler-based translations, which are verifiable but incomplete, into the prompts given to the LLMs, thereby improving code translation without requiring additional resource-intensive fine-tuning of pretrained models. To evaluate this method, we construct TGAlpaca, a new dataset comprising Java, JavaScript, and Swift snippets curated from programming tutorials and the HuggingFace Alpaca repository. Two instruction-tuned, code-specific LLMs (Codestral 25B and Qwencoder 32B), as well as the general-purpose GPT-4o-mini, are benchmarked under baseline and transcompiler-assisted (TC-assisted) prompting conditions. On TGAlpaca, our evaluation shows consistent improvements, from as little as 2% in Java→Swift to as high as 40% in JavaScript→Java, illustrating the potential of our approach. We further assess the methodology on XLCoST dataset using Codestral 25B model, its fine-tuned version, and its TC-assisted version. We observe that TC-assisted prompting can increase translation fidelity, reduce dependence on costly fine-tuning, and strengthen trust in LLM-assisted software engineering workflows.

Authors' Contact Information: Amira T. Mahmoud, AmTarek@nu.edu.eg, Center for Informatics Science (CIS), School of Information Technology and Computer Science, Nile University, Giza, Egypt; Nahla Elaraby, Institute of Computer Technology, Technical University of Vienna (TU Wien), Gusshausstrasse 27-29, 1040 Vienna, Austria and Canadian International College-CIC, Cairo, Egypt; Hala Zayed, Faculty of Engineering, Egypt University of Informatics, Cairo, Egypt and Faculty of Computer and Artificial intelligence, Department of Information Systems, Benha University, Cairo, Egypt; Ahmed H Yousef, El Sewedy University of Technology, Cairo, Egypt and Ain Shams University, Faculty of Engineering, Computers & Systems Department, Cairo, Egypt; Walaa Medhat, Center for Informatics Science (CIS), School of Information Technology and Computer Science, Nile University, Giza, Egypt and Faculty of Computer and Artificial intelligence, Department of Information Systems, Benha University, Cairo, Egypt.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

CCS Concepts: • **Software and its engineering** → **Source code generation**; **Software notations and tools**; **Compilers**; • **Computing methodologies** → **Machine translation**;

ACM Reference Format:
Amira T. Mahmoud, Nahla Elaraby, Hala Zayed, Ahmed H Yousef, and Walaa Medhat. 2025. Building Trust in LLM-Based Code Translation through Transcompiler-Assisted Prompting. 1, 1 (December 2025), 21 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Code translation is a fundamental task in software engineering and offers substantial benefits for modernization, portability, and cost reduction. Migrating legacy systems written in outdated or less efficient languages to contemporary ones can improve the performance, maintainability, and long-term sustainability of software systems [6, 27, 30]. Translation across programming languages also enables applications to run on multiple platforms, a necessity in today's heterogeneous computing environments [3]. One important example is mobile application development, where cross-platform frameworks are widely used but developers still rely on platform-specific languages when specialized functionality or high performance is required [10, 26]. In such cases, automated code translation to native languages can help bridge the gap by delivering near-native performance without extensive manual recoding, thereby reducing both development effort and cost.

Large language models (LLMs) have recently demonstrated good capabilities in code generation and translation [14, 19, 31], driven by large-scale pretraining on extensive monolingual code corpora; collections of code written in a single language. However, for accurate translation across languages, bilingual or parallel datasets, pairs of semantically equivalent code snippets in two different languages, are required to fine-tune models for cross language alignment. Fine-tuning in this context refers to the process of adapting a pretrained model to specific translation tasks using such bilingual data to capture consistent syntax and semantics across languages. Unfortunately, high-quality bilingual datasets for code are scarce, especially for domain-specific or less common languages such as Dart, Kotlin, and Swift. For example, CodeTransOcean [47] remains the only public benchmark containing Swift–Java code pairs, with only about 1.3k examples collected from Rosetta Code. Moreover, many of these pairs lack strict syntactic and semantic equivalence, limiting their value for fine-tuning models such as StarCoder, CodeT5, or CodeBERT. As a result, LLMs often struggle to generate translations that are both syntactically correct and functionally faithful, undermining developer trust and underscoring the need for alternative strategies that improve reliability without relying on scarce parallel datasets.

In recent years, instruction-tuned models have emerged widely. Fine-tuning instruction tuned models with monolingual code datasets appeared as an alternative to training with bilingual datasets [7, 41, 45]. These models are not trained for a single specialized code task; instead, they are adapted to follow natural language instructions while leveraging large monolingual code corpora. As a result, instruction-tuned models such as Codestral and Qwencoder can generalize across a range of software engineering tasks, including code summarization, generation, and code translation between programming languages.

Incorporating compiler semantics into LLMs, through methods such as semantic facts augmentation, intermediate representations, and compiler-based tokenization, significantly enhances their ability to translate or generate code [2, 21, 37, 44]. Previous approaches have always used an intermediate representation of the input code. Still, to our knowledge, no previous work has proposed including parts or representations of the output expected code.

In this work, we present a new hybrid approach that integrates transcompilers, previously applied to mobile application code translation [4, 11, 16, 25, 27–29, 34, 35], with code-specific LLMs (Codestral and QwenCoder) and Generic LLMs(GPT4o). This integration seeks to combine the verifiability of compiler outputs with the adaptability of

LLMs, thereby improving both translation accuracy and developer trust. The trans-compilers understudy generate base translations for Java-Swift, Swift-Java, JavaScript-Java, and JavaScript-Swift translation paths. These base translations are verifiable but incomplete due to trans-compilers' limitations [23]. Incomplete translations act as a guide to the prompted LLMs to reduce hallucination and improve code translation accuracy.

The main contributions of this paper are as follows:

- Introducing a hybrid methodology, TC-assisted prompting, that integrates between transcompiler-based code translators and Large language models for translating between programming languages with scarce parallel datasets.
- Providing a new monolingual curated dataset for execution-based evaluation, named TGAlpaca, with a total of 902 code snippets for the programming languages Java, Swift, and JavaScript.
- Evaluation of two newly released code generation models, QwenCoder 32B and Codestral 25B, alongside the general model GPT-4o-mini on code translation tasks.
- Fine-tuning the Codestral model with the XLCoST dataset and applying the TC-assisted methodology to compare our approach with the traditional fine-tuning approach.
- Empirical analysis of CodeBLEU in the context of LLM-based code translation, confirming prior findings that surface-level metrics can be misleading [32], and demonstrating within our experimental settings that execution-based evaluation provides a more reliable assessment of functional correctness.

The remainder of this paper is structured as follows. Section 2 reviews the relevant background and related work. Section 3 details the proposed TC-assisted prompting methodology, including the model selection strategy and experimental setup. Section 4 describes the datasets used in this study, including the curated TGAlpaca dataset and the benchmark XLCoST dataset. Section 5 outlines the fine-tuning experiments conducted with the Codestral model. Section 6 introduces the evaluation metrics employed to assess translation quality. Section 7 presents and analyzes the experimental results of the proposed integrated approach, followed by a discussion of the threats to validity in Section 8. Finally, Section 9 concludes the paper and outlines potential directions for future work.

2 Background and Literature Review

Initial research in code translation leveraged Neural Machine Translation (NMT) techniques, which applied sequence-to-sequence models to translate code snippets. These models demonstrated improved performance over traditional methods by learning from large datasets of code pairs. However, they were limited by the availability of high-quality parallel code datasets. [5, 9, 17].

The introduction of transformer architectures marked a significant advancement in code translation, which represents a breakthrough in the world of NLP. The architecture of translation models became more complex using transformers with attention [38]. Alongside this breakthrough, large data availability has introduced LLMs to our world. Many models were trained for code-specific tasks such as Trans-coder[36], Bigcoder, CodeT5 [43], Star coder[19]. These models were trained with software programming languages such as C#, Java, Python, and C++. Fewer models are trained with languages like Swift, Kotlin, and JavaScript. In the following subsections, we will present the latest research that aimed at advancing LLMs for better code generation and translation. Then, the last subsection presents a background on transcompiler-based code translation.

2.1 Tuning LLMs for enhanced code generation

Several significant challenges face LLMs for code generation and translation, especially when dealing with scarce parallel code pairs [50, 51]. The limited availability of high-quality parallel code data hinders the ability of models to learn precise cross-language alignments. Although synthetic parallel code data can be generated to mitigate this issue, ensuring the quality and alignment of such data remains a challenge. Poorly aligned synthetic data can lead to translation errors, reducing the overall effectiveness of the model. Efficient techniques for tuning LLMs with limited data have emerged, such as parameter-efficient tuning, which updates only a small set of parameters while reusing prior knowledge. However, these methods still require careful optimization [20, 46]. Another solution to improve the performance of LLMs in code generation tasks is prompt tuning [39, 40]. This approach involves inserting trainable embeddings into the inputs of pre-trained language models (PLMs) while keeping the model parameters frozen, which significantly reduces the computational resources required compared to traditional fine-tuning methods, yet these approaches still demand careful optimization and significant computational resources.

2.2 Code in the Middle for code to code translation

Recent work explores transitive code translation, instead of translating directly between a source and target language. INTERTRANS [22] plans and executes intermediate hops (e.g., Python->C++->Java) using an LLM, then validates each candidate target with a test suite. A Tree of Code Translations enumerates candidate paths, with caching and early stopping to reduce redundant inference. INTERTRANS reports substantial gains over direct prompting from 18.3% to 43.3% absolute Computational Accuracy and achieves up to 93.8% CA on TransCoder, outperforming strong baselines. Although the method is LLM-agnostic and requires no retraining, it trades the extra inference cost for accuracy, and its gains depend on the choice of intermediate-language. This method is close to our method in the idea of two-step code translation. However, they use programming languages in the middle that are different from the target language, and depend on exploring the different paths through LLM translations. In our case, we have only one representation in the middle, which is part of the expected code in the target language generated by a transcompiler.

2.3 Compiler-assisted LLMs for Code Generation

A different line of work augments LLMs with compiler semantics and techniques to improve code translation quality. CoTran [18] proposes an end-to-end, whole-program translation framework that fine-tunes an LLM with reinforcement learning using compiler feedback and symbolic execution-based testing signals to steer the model toward compilable and functionally equivalent output. Evaluated on 57k Java->Python pairs against 14 baselines, CoTran reports state-of-the-art compilation accuracy and functional equivalence accuracy.

Other work addressed low-resource programming languages by bootstrapping training data and using compiler feedback in the loop [42]. Starting from monolingual seed corpora, the method back-translates to synthesize parallel datasets and then refines the model with compiler-driven signals. As a case study on the new low-resource language Cangjie, the authors built a Java->Cangjie parallel test set and accompanying test cases, showing that compiler feedback markedly improves syntactic validity, semantic fidelity, and test pass rates of the translated code. These results highlight a practical route for learning-based translation in settings with limited parallel data. Another thread leverages low-level intermediate representations like LLVM IR to expose language-agnostic program structure and semantics, thus improving translation accuracy [37].

Compiler information was also used for other code generation tasks, like code completion and code summarization. For example, Li et al. [21] used compiler-aware tokenization to preprocess the source code, shrinking the input length while preserving the correct structure for code completion. Their experiment reported a 33.7% reduction in tokens and higher-quality embeddings, enabling better code completion with CodeLlama-7B without sacrificing accuracy. Prompt-level enrichment has also shown benefits through injecting semantic information such as parameter names, return expressions, and control-flow summaries. The results produced favorable changes by 44% for code summarization cases across Codex, GPT-3.5, and Code-Davinci [2]. Our approach is closest in spirit to this last category: we provide the LLM with a partial translation generated by the transcompiler, embedding it directly in the prompt to guide the final output of the model. A summary of compiler-assisted methods is shown in Table 1.

Table 1. Summary of Compiler-Assisted Enhancements in LLM-Based Code Generation Tasks

Method	Coding Task and Languages	Key Results
LLM with reinforcement learning using compiler feedback [18]	Code translation: Python, Java	Improved translation accuracy by 8.14% CompAcc and 14.89% FEqAcc for Python->Java; 4.30% and 12.94% for Jav->Python over CodeT5 backbone
Compiler feedback in the loop to synthesize parallel datasets [42]	Code translation: Cangjie, Java	the overall performance of the method model is significantly superior to that of baselines, achieving around 20% improved pass rate
Using low-level compiler intermediate representations (IRs) [37]	Code translation: C++, Java, Rust, Go	Improved translation accuracy by 11% using LLVM IR and TransCoder as baseline.
Augmenting prompts with semantic and syntactic code information [2]	Code summarization: Java, JavaScript, Go, Python, Ruby, PHP	Induced positive changes in 44% of samples using Codex, GPT-3.5, and Code-Davinci.
Compiler-based input token compression [21]	Code completion: Java	Reduced token length by 33.7% using CodeLlama 7b while maintaining model accuracy.

2.4 Trans-compiler-based code translators

Previous research that utilized compilers to enhance LLMs for code translation employed compiler information about the input code or the intermediate representation of the input code. None has provided a real translation that shows parts of the expected output. This idea would change the problem to guided code translation, which is much simpler for the LLM, given parts of the code to be generated. Code-to-code translation using a trans-compilation approach has been tackled previously by our research group for Swift, Java, and JavaScript code translation [4, 11, 16, 25, 27–29, 34, 35]. Other source-to-source translation tools—commonly referred to as transpilers or trans-compilers—have also been widely studied and adopted in both research and industry. A recent systematic mapping study [**?**] reports that transpilers are used across multiple domains, including legacy system migration, cross-platform development, web application modernization, and safety-critical software re-engineering. Practical tools such as C2Rust, cxgo, Sharpen, and Java-to-C# converters demonstrate sustained interest in automated code translation, even though such tools are often limited to specific language pairs. While Trans-compilation preserves code structure and correct semantics, it is still not a complete approach, and sometimes is not capable of direct translation using the compiler rules for conversion. Therefore, in

this paper, we evaluate integrating the trans-compiler-based translation to the prompt given to the instruction-based LLMs to get a base translation. Then, we prompt the LLMs to complete this translation. We evaluate how the partial compiler-based translation given in the prompt will leverage the LLMs' ability to produce more accurate translations for the given input codes.

3 TC-assisted-prompting Methodology

The TC-assisted-prompting methodology presented in this section is model-agnostic and can be applied to any instruction-based LLM. Instruction-based LLMs are models trained to follow natural-language instructions. Figure 1 illustrates the core workflow of the TC-assisted prompting methodology. The process begins with a set of input code snippets, which are passed through the transcompiler-based translator. The transcompilers apply grammar-rule-based conversions and produce partially complete yet semantically structured compiler-translated code snippets. These outputs often lack support for certain language constructs or edge cases, but provide a solid structural basis for further completion [23]. The compiler-translated snippets are then used as a trusted context in the prompt fed to the LLM. Through careful prompt engineering, the LLM is instructed to complete the translation using the partially verified output from the transcompiler. This results in prompt-assisted translated code snippets, which benefit from the combined strengths of formal translation logic of transcompilers and the generative flexibility of LLMs.

To assess the TC-assisted prompting approach, we evaluate three instruction-tuned LLMs, two code-centric models (Qwen-Coder-32B and Codestral-25B), and a general baseline commonly used for code generation (GPT-4o mini).In the next subsections, we first justify the selection of the models and describe their relevant capabilities. Second, we detail a two-step experimental protocol used to compare performance across models.

3.1 Models Selection Strategy

A wide range of models could be used to apply our proposed methodology. To identify the most suitable candidates, we queried the HuggingFace Open LLM Leaderboard [15], filtered by the keyword `code`, restricted results to `official providers only`, and excluded flagged entries such as unavailable or contaminated models. According to these criteria, Qwen-Coder-32B was the highest-ranked model, followed by Qwen-Coder-14B and Codestral-22B. Since Qwen-Coder-32B supersedes the 14B version, we excluded the smaller variant. For the Codestral family, we used the more recent Codestral-25B release, which replaced Codestral-22B on the Mistral AI platform. Leaderboard rankings are derived from six evaluation metrics, including factual knowledge, reasoning (both short and long context), mathematics, and instruction-following capabilities that closely align with code translation requirements. This leaderboard-based selection was performed in February 2025. In addition to these code-specialized models, we included GPT-4o mini, a widely used general-purpose model known for strong performance on code generation tasks.

To maintain a low-barrier experimental setup, we prioritized models with accessible API endpoints rather than requiring local GPU hosting. Codestral-25B was accessed through the Mistral AI platform, which provided open API access at the time of experimentation, while Qwen-Coder-32B and GPT-4o mini were accessed via OpenRouter[1] , a unified API gateway for both open and commercial LLMs. This API-driven configuration enables a cost-effective and scalable evaluation pipeline.

[1] https://openrouter.ai

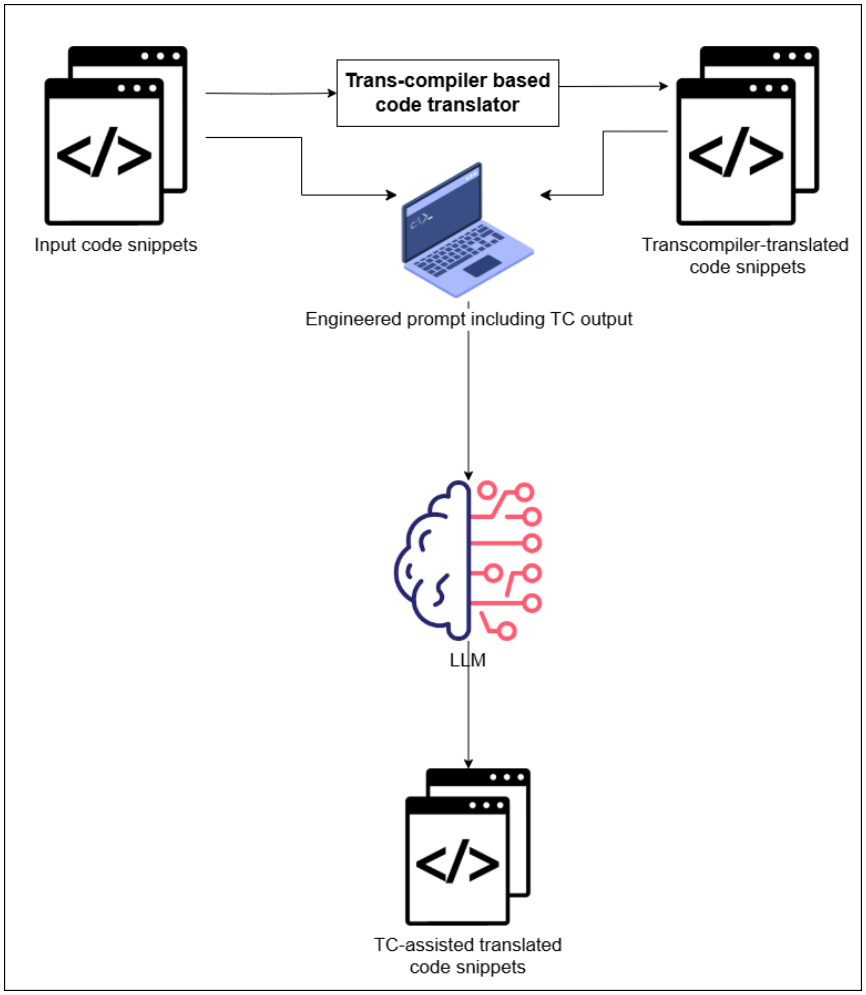

Fig. 1. TC-assisted prompting methodology

3.1.1 Rationale for Selecting Codestral-25B for XLCoST Dataset. In our evaluation, we will use two main datasets: our curated dataset, named TGAlpaca, and the benchmark dataset XLCoSt. The Codestral-25B model was chosen as the primary model for further benchmarking on XLCoST. This decision was driven by three main considerations:

- **Accessibility:** Codestral provided open and reliable API access through the Mistral platform, whereas GPT-4o and Qwen-Coder required restricted or paid access, limiting large-scale experimentation.
- **Performance:** In the TGAlpaca experiments, TC-assisted Codestral achieved the highest Execution Output Match (EM) score for the JavaScript→Java translation task—the same translation direction evaluated in XLCoST.
- **Fine-tuning Capability:** The Mistral platform uniquely enables cloud-based fine-tuning of the Codestral family of models. This feature allowed us to perform controlled fine-tuning experiments on the XLCoST benchmark to directly compare the effectiveness of TC-assisted prompting with conventional fine-tuning, even under limited computational resources.

These considerations made Codestral-25B the most practical and representative model for assessing TC-assisted prompting with a dataset benchmark like XLCoST.

3.2 Two-step Experimental Setup

Figure 2 summarizes a two-step evaluation built on the pipeline in Figure 1. We start with a set of input code snippets and feed them to the *transcompiler-based translator* to obtain partially structured base translations.

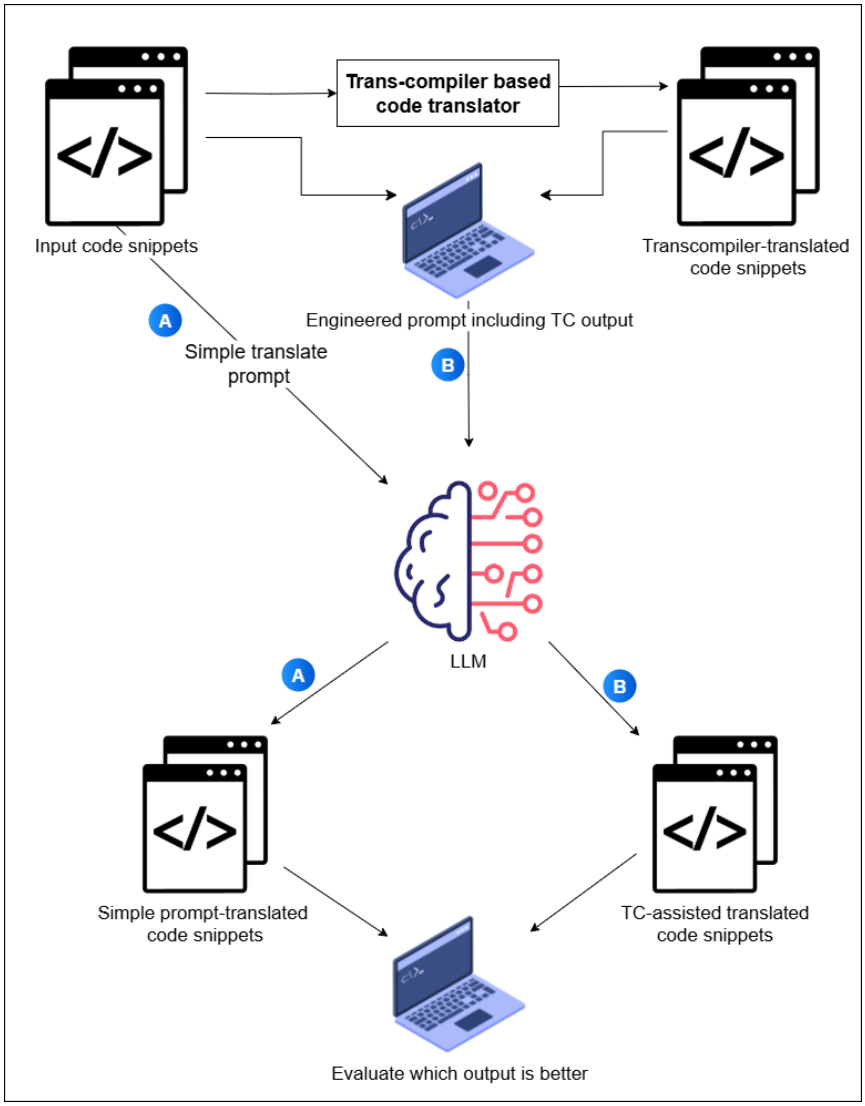

Fig. 2. Two-step experiment for evaluating TC-assisted prompting.

(A) Unguided LLM translation. The same inputs are sent directly to the LLM with a *simple translate prompt* that provides no compiler-derived guidance. The result is a set of LLM-translated snippets reflecting the model's

raw translation ability. These outputs can be semantically reasonable but may deviate structurally or include hallucinated elements. A sample prompt:

```
You are a code translator. Translate the given Swift code to Java code. Give the
resulting code between quotes. The Swift code:"swift_code"
```

(B) TC-assisted LLM translation. As described in Section 3, we prompt the LLM again, this time supplying the transcompiler-produced base translation as trusted context to guide the final output. A sample prompt is shown below:

```
You are a code translator. Translate this Swift code to Java. Stick closely to the
provided Java base translation. Make minimal changes only to complete the code. In
case the Java base is empty or has failed conversion, translate the Swift code
yourself. Enclose the corrected translation between quotes.
Swift code:
"swift_code"
Base Java translation:
"base_java_code"
```

For each model, we therefore obtain two outputs for comparison: (i) the plain LLM result from the simple prompt and (ii) the TC-assisted result.

4 Datasets used for evaluating TC-assisted prompting

We evaluated our approach using two datasets: (1) the custom-curated TGAlpaca dataset, (2) the XLCoST benchmark dataset. The datasets include code snippets from three programming languages—Swift, Java, and JavaScript—commonly used in mobile application development. These languages were selected due to the availability of transcompilers supporting the following translation directions: Swift → Java, Java → Swift, JavaScript → Java, and JavaScript → Swift.

4.1 TGAlpaca Dataset

The TGAlpaca dataset contains 262 Swift code snippets manually collected from popular online programming tutorials such as TutorialsPoint and GeeksforGeeks. The snippets span a wide range of language features, from basic constructs like variables, loops, and conditions, to more advanced object-oriented programming concepts. These examples were selected to ensure comprehensive coverage of the translation rules we verified in previous work [23].

In addition, we collected 441 Java code snippets from the same online sources, along with the Alpaca Code dataset available on HuggingFace[2] . We filtered the Java snippets to include only those that compiled successfully. Although the Java-to-Swift transcompiler wasn't formally verified in our prior work, it shares the same rule-based architecture as the Swift-to-Java transcompiler. As such, the verification methods apply to both, as well as to the JavaScript-to-Java and JavaScript-to-Swift transcompilers tested in this paper.

The dataset also includes 199 JavaScript code snippets, selected to evaluate the JavaScript-to-Java and JavaScript-to-Swift transcompilers. The dataset sources and statistics are provided in Table 2. The TGAlpaca dataset serves as a focused and practical testbed to assess how verified transcompiler outputs can guide LLMs to produce more accurate and semantically valid code translations.

[2] https://huggingface.co/datasets/amztheory/alpaca-code-java

Table 2. TGAlpaca Dataset samples count and Data Sources

Translation Source Language	Number of Code Snippets	Data Source(s)
Swift	262	TutorialsPoint, GeeksforGeeks (manually collected)
Java	441	TutorialsPoint, GeeksforGeeks (manually collected), HuggingFace (filtered Alpaca Code)
JavaScript	199	TutorialsPoint, GeeksforGeeks

4.2 XLCoST Dataset

The XLCoST dataset [49] is also utilized for the JavaScript-to-Java translation task. XLCoST is one of the largest parallel datasets for code, containing over one million parallel code snippets in seven programming languages, including Java and JavaScript. It offers both snippet-level and program-level alignment, making it highly suitable for evaluating code generation and translation models. We used 8,471 training pairs for the fine-tuning experiment that will be detailed in Section 5, and 881 test pairs for evaluation and comparison.

5 Codestral Fine-tuning Experiment

We evaluate fine-tuning as a comparator to TC-assisted prompting using the JavaScript→Java pairs from the XLCoST dataset. Our study considers four scenarios, summarized in Figure 3:

- **(A) Fine-tuned baseline:** Fine-tune Codestral-25B on the XLCoST training split (8,471 pairs) and evaluate on the test split (881 pairs).
- **(B) Fine-tuned + TC:** Apply the TC-assisted prompting method to the fine-tuned model on the same test split.
- **(C) Zero-fine-tune, simple prompt:** Use Codestral-25B with a simple translation prompt on the test split.
- **(D) Zero-fine-tune + TC:** Apply TC-assisted prompting to Codestral-25B on the same test split.

For all conditions, generated Java code is compared to XLCoST references using CodeBLEU, enabling direct comparison with benchmark results reported for CodeBert, PlBert,CodeT5, and the CodeGeex model. We will also use execution based evaluation for the same set of compared models. As noted in the XLCoST paper [49], the CodeBert, PlBert, and CodeT5 baselines were trained on four RTX 8000 GPUs (48,GB each).

We fine-tuned Codestral-25B via Mistral's *La Plateforme* using their text fine-tuning workflow. The configuration specified a learning rate of 1×10^{-5} and 5 epochs; the platform determined the number of training steps based on dataset size and an internal batch configuration (not user-exposed), yielding 148 observed steps. The end-to-end fine-tuning run completed in approximately 35 minutes.

6 Evaluation Metrics

Our evaluation focuses on metrics that measure both the syntactic correctness and the functional equivalence of translated code. Although metrics such as CodeBLEU [12] surface-level similarity between a generated translation and a reference, they do not assess whether the translated code is correct or semantically equivalent to the source. We use CodeBLEU here to contextualize our results against prior studies and to highlight the advantages of execution-based evaluation to assess true functional equivalence in code translation. Below, we explain each evaluation metric used.

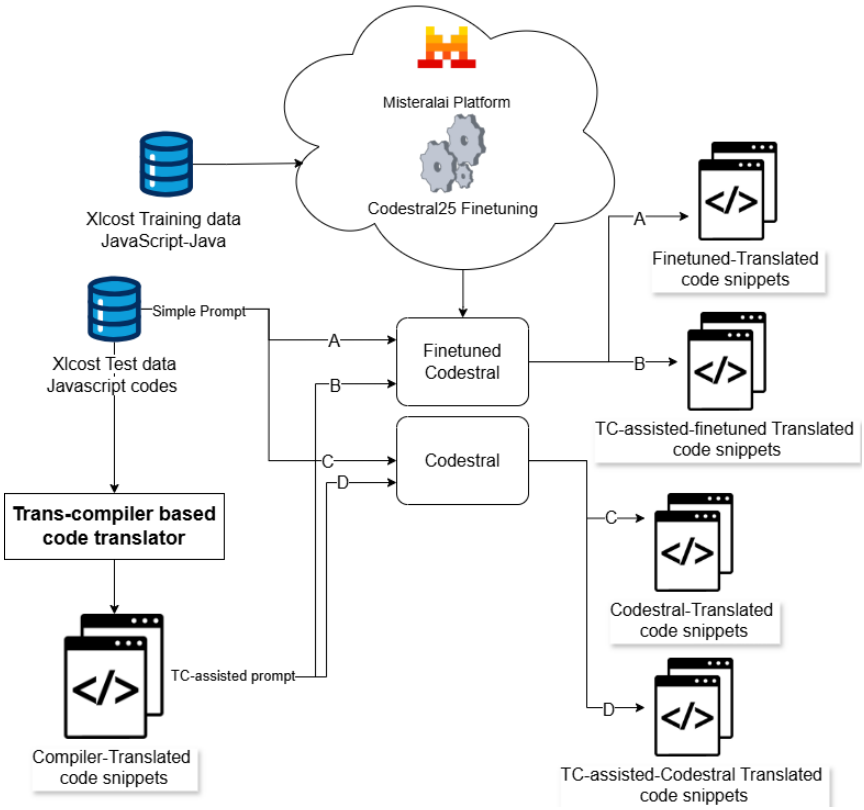

Fig. 3. Four setups for Codestral model: Fine-tuned Codestral, Fine-tuned TC-assisted Codetsral, plain Codetsral model, and TC-assisted Codestral

6.1 Exact Output Match Score

To assess functional equivalence, we employ an execution-based evaluation approach. In this setup, each program is executed, and its output is compared against the expected output. Minor formatting differences (e.g., whitespace or newline variations) are normalized before comparison. We compute the Execution Output Match Score (EM) (Equation 1) as the percentage of translations that produce outputs identical to their reference implementations. This metric captures semantic fidelity beyond syntactic similarity.

$$\text{Execution Output Match Score (EM)} = \frac{\text{\# of correctly matched outputs}}{\text{Total \# of translations}} \times 100 \tag{1}$$

6.2 Compilation Success Rate

The Compilation Success Rate (CR), measures the syntactic validity of the generated code by verifying whether each translation compiles successfully in the target language. For each output, we attempt to compile it using the appropriate

compiler and compute the CR as the proportion of compilable outputs among all translations, as shown in Equation 2.

$$\text{Compilation Success Rate (CR)} = \frac{\text{\# of successfully compiled translations}}{\text{Total \# of translations}} \times 100 \tag{2}$$

While a high CR indicates that the model can produce syntactically correct code, it does not guarantee that the output is functionally equivalent to the source. In fact, the difference between CR and the Exact Match (EM) score reveals cases where the model "hallucinates" — generating code that successfully compiles but performs a different or incomplete function compared to the intended translation. Thus, CR helps identify when the model favors producing compilable code over preserving semantic correctness.

6.3 CodeBLEU

While our primary evaluation emphasizes execution-based metrics (i.e., compilation success and execution output match), we include CodeBLEU for comparative purposes, particularly because models evaluated on the XLCoST benchmark relied on it. Originally introduced by Ren et al. [33], CodeBLEU is an automatic metric designed for code synthesis and translation. It builds on the traditional BLEU metric, which measures n-gram overlap, by additionally incorporating structural and semantic features of code, namely Abstract Syntax Tree (AST) similarity and data-flow (DF) similarity.

Formally, if we denote:

- S_{ng} = n-gram match score,
- S_{ast} = AST match score,
- S_{df} = data-flow match score,
- S_{kw} = keyword match score,

then CodeBLEU is computed as a weighted sum:

$$\text{CodeBLEU} = \lambda_1 \cdot S_{\text{ng}} \;+\; \lambda_2 \cdot S_{\text{ast}} \;+\; \lambda_3 \cdot S_{\text{df}} \;+\; \lambda_4 \cdot S_{\text{kw}}, \quad \text{with} \sum_{i=1}^{4} \lambda_i = 1.$$

In code-translation studies, CodeBLEU has been used extensively as a key metric for evaluating model output quality. However, recent analyses argue that despite its enhancements over BLEU, CodeBLEU still focuses on syntactic and surface similarity rather than verifying functional correctness or runtime behaviour [12]. Consequently, while CodeBLEU remains valuable for benchmarking and comparison, it does not guarantee that a generated translation will compile or execute correctly—justifying our emphasis on execution-based evaluation.

7 Results

This section presents the evaluation results of the TC-assisted prompting methodology. We first report experiments on the TGAlpaca dataset across seven configurations. The evaluation uses the Compilation Success Rate (CR) and the Execution Output Match Score (EM), which assesses functional equivalence. We follow this experiment by a focused evaluation of the TC-assisted Codestral model, its plain version and its fine-tuned version on XLCoST. We have selected Codestral model only for the benchmark dataset due to resource limitations and performance advantage as explained in Section 3.1.1.

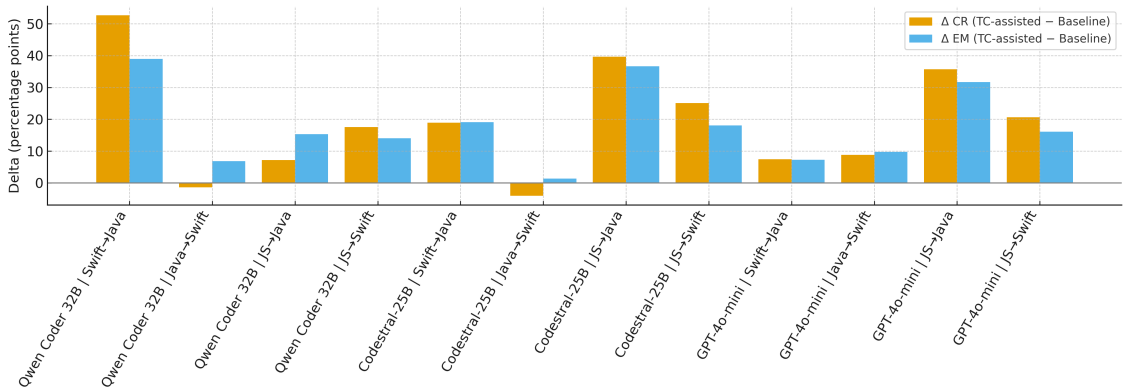

Fig. 4. Improvement (ΔCR and ΔEM) from baseline to TC-assisted across models and translation directions.

7.1 Results on the TGAlpaca Dataset

The initial experiment on TGAlpaca was conducted across six configurations. For each of the three evaluated models—GPT-4o-mini, Qwen-Coder-32B, and Codestral-25B, two configurations were tested: (1) a baseline translation using a simple prompt without compiler guidance, and (2) a TC-assisted translation where the partial transcompiler output was embedded into the prompt to guide the model's generation. Table 3 shows the CR and EM of the six configurations.

Table 3. Compilation Success Rate (CR) and Execution Output Match (EM) on the TGAlpaca dataset across four translation directions for each of the three tested models and their TC-assisted versions. Scores are in %.

	Swift→Java (262)		**Java→Swift (441)**		**JS→Java (199)**		**JS→Swift (199)**	
Model	**CR**	**EM**	**CR**	**EM**	**CR**	**EM**	**CR**	**EM**
Qwen Coder 32B	11.07	10.31	56.23	23.80	19.42	6.31	15.07	9.04
TC-assisted Qwen Coder 32B	63.74	49.24	54.87	30.61	26.63	21.61	32.66	23.11
Codestral-25B	63.86	44.66	78.45	38.54	41.21	31.66	33.16	24.62
TC-assisted Codestral-25B	82.82	**63.74**	74.37	**39.90**	80.90	**68.34**	58.29	42.71
GPT-4o-mini	70.41	48.09	55.55	26.75	48.74	33.67	49.74	31.65
TC-assisted GPT-4o-mini	77.86	55.34	64.39	36.50	84.42	65.33	70.35	**47.73**

We evaluated four translation directions: Swift→Java, Java→Swift, JavaScript→Java, and JavaScript→Swift. As mentioned before, only these directions are included as they are the ones covered by the transpiler understudy. Table 3 shows the corresponding CR and EM scores for the six tested configurations. Since we are interested in quantifying the improvements achieved through TC-assisted prompting, Figure 4 presents the relative deltas in Execution Match (ΔEM) and Compilation Rate (ΔCR) across all translation scenarios and models.

The results indicate consistent improvements in Exact output match scores (EM) when TC assistance is applied. The most notable EM improvement is 40% observed for the Qwen-coder model for Swift→Java translation. The minimal EM improvement is about 2% between Codetsral and its TC-assisted version for the Java→Swift translation task. Observing the Compilation success rate, we can also see consistent improvements, except for the Java→Swift translation task, we can see that Codestral plain model achieved higher CR than the TC-assisted model by 4% and QwenCoder plain model also got higher CR than TC-assisted Qwen. By observing the EM scores for the same translation task we can find almost

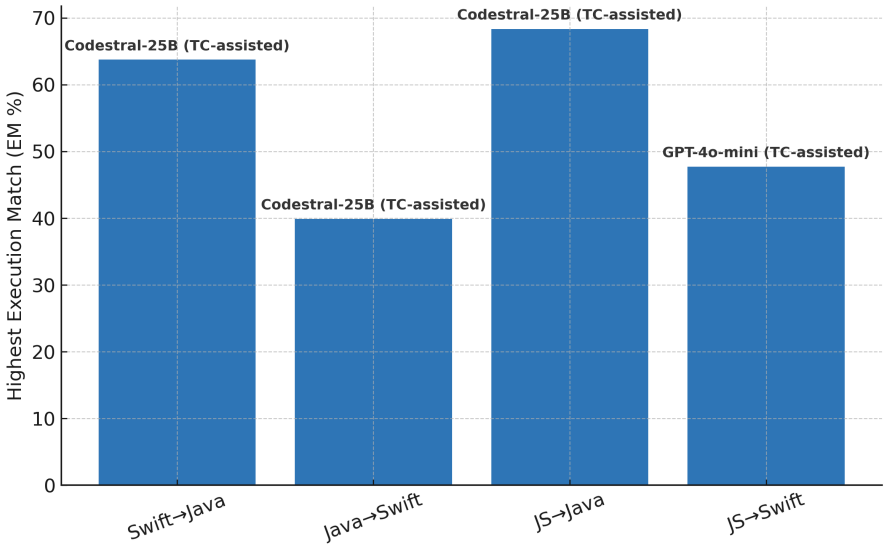

Fig. 5. Highest achieved Execution Match (EM) score across all models and configurations for each translation direction. The labels indicate the model and configuration achieving the top accuracy.

equal EM score between Codetsral and its TC-assited version and higher EM for the TC-assisted Qwen model than the plain one by almost 7%. This means that although the TC-assised models in these cases produced more syntactic errors for Java→Swift traslation, they still didn't reduce the semantic equivalence since the EM scores improved by almost 2% for Codetsral and 7% for Qwen.

To identify which model–language pair combinations achieved the highest translation accuracy, Figure 5 summarizes the top Execution Match (EM) scores obtained across all configurations for each translation direction. The TC-assisted Codestral-25B model consistently delivers the best results across most translation paths, achieving the highest EM scores for Swift→Java and JavaScript→Java with improvements of 19% and 36%, respectively, over the plain Codestral model. These results indicate that the compiler-guided prompting strategy is particularly beneficial for language pairs with substantial syntactic and structural differences. Meanwhile, the TC-assisted GPT-4o-mini model achieves the best EM score for JavaScript→Swift, improving by 16% over its baseline counterpart. In contrast, the Java→Swift direction shows the lowest EM scores and the smallest relative improvement, reflecting the greater difficulty of translating from a strongly typed, verbose language such as Java into Swift, which, despite being statically typed, emphasizes brevity and expressive constructs. Overall, these findings demonstrate that the effectiveness of TC-assisted prompting varies by translation direction, with the largest benefits observed in tasks involving greater linguistic and paradigm divergence between source and target languages.

In general, a closer inspection of compilation logs revealed that syntax-related errors (e.g., illegal tokens or missing delimiters) were notably reduced in TC-assisted outputs. Instead, most remaining errors were semantic, due to type mismatches, symbol resolution, or logical inconsistencies, indicating that the generated code was structurally valid and closer to the target language conventions.

7.1.1 Illustrative Example. An illustrative example is shown in Table 4, demonstrating the benefit of TC-assisted prompting in preserving structural and semantic fidelity during translation. The example involves a Java program that exemplifies fundamental object-oriented principles such as encapsulation and controlled access through getter and setter methods. The original Java code defines a `Person` class with private fields (`name` and `age`) and public accessors. A separate `Geeks` class contains the `main` method, which instantiates `Person`, sets field values, and prints the output. This example serves as a straightforward yet representative case for assessing both syntactic and behavioral accuracy in cross-language translation between Java and Swift.

Table 4. Java→Swift sample translation comparison between QwenCoder plain model and TC-Assisted QwenCoder.

Java Input

```
class Person {
  private String name;
  private int age;

  public String getName() {
     return name; }
  public void setName(String name
     ) {
    this.name = name;
  }

  public int getAge() { return
     age; }
  public void setAge(int age) {
    this.age = age;
  }
}

public class Geeks {
  public static void main(String
     [] args) {
    Person p = new Person();
    p.setName("John");
    p.setAge(30);
    System.out.println("Name: " +
      p.getName());
    System.out.println("Age: " +
     p.getAge());
  }
}
```

Qwen Output (Swift)

```
class Person {
  private var name: String
  private var age: Int

  init() {
    self.name =
    self.age = 0
  }

  func getName() -> String {
     return name }
  func setName(_ name: String) {
     self.name = name }
  func getAge() -> Int { return
     age }
  func setAge(_ age: Int) { self.
     age = age }
}

struct Geeks {
  static func main() {
    var p = Person()
    p.setName("John")
    p.setAge(30)
    print("Name: \(p.getName())")
    print("Age: \(p.getAge())")
  }
}
Geeks.main()
```

TC-assisted Qwen (Swift)

```
class Person {
  private var name: String = ""
  private var age: Int = 0

  public func getName() -> String
      { return name }
  public func setName(name:
     String) { self.name = name }
  public func getAge() -> Int {
     return age }
  public func setAge(age: Int) {
     self.age = age }
}

class Geeks {
  public func main(args: [String
     ]) {
    let p = Person()
    p.setName(name: "John")
    p.setAge(age: 30)
    print("Name: ", p.getName())
    print("Age: ", p.getAge())
  }
}
let mainInstance = Geeks()
mainInstance.main(args: [])
```

When translated using the baseline Qwen model, several issues arise. The generated Swift code fails to properly initialize class properties in the `init()` method, omitting the `name` parameter entirely. Moreover, the model incorrectly converts both `Person` and `Geeks` into Swift `structs`, diverging from the object-oriented paradigm of the original Java program. This structural shift, along with minor syntax inconsistencies (such as incorrect print statement escaping), undermines semantic fidelity and results in non-compilable output. By contrast, the TC-assisted Qwen translation

produces a more faithful and executable Swift version. The `Person` entity is correctly represented as a Swift `class`, with properly initialized properties and idiomatic getter/setter methods using parameter labels. The `Geeks` class and its `main()` method are preserved accurately, maintaining both the program structure and runtime behavior. This example illustrates how TC-assisted prompting mitigates typical LLM translation errors by grounding the generation process in verified rule-based transcompiler output. Even when the initial translation of the transcompiler is incomplete, it provides the LLM with a syntactically and semantically stable foundation. The resulting output adheres more closely to the conventions and semantics of the target language, reinforcing the effectiveness of combining compiler-driven structure with instruction-tuned LLM reasoning.

7.2 Results of XLCoST dataset

This section reports the results of applying the TC-assisted methodology to Codestral-25B model on the XLCoST benchmark dataset. As explained in section 3.1.1, we have chosen the Codestral model only with benchmarked data due to resource constraints. We used CodeBleu and Execution-based evaluation to compare both evaluation metrics on the benchmark models, fine-tuned Codetsral, and TC-assisted Codestral using the JavaScript→Java subset of the XLCoST dataset [49]. The XLCoST test set includes 881 aligned JavaScript–Java code pairs, each comprising a source snippet and its corresponding reference translation. In addition, we investigate the effect of fine-tuning the Codestral-25B model on the same dataset to examine and compare training-based and prompt-based enhancements.

Figure 6 summarizes the CodeBLEU results across four configurations: the base Codestral-25B model, its TC-assisted variant, a fine-tuned version trained on the XLCoST training set, and the TC-assisted fine-tuned model. For broader context, benchmark results from prior models—CodeBERT [13], PLBART [1], CodeT5 [43], and CodeGeex [48]—are also included. While the fine-tuned Codestral-25B achieves the highest CodeBLEU score (41.19) among our configurations, it remains considerably lower than the fine-tuned benchmark models, which exceed 84 on this metric.

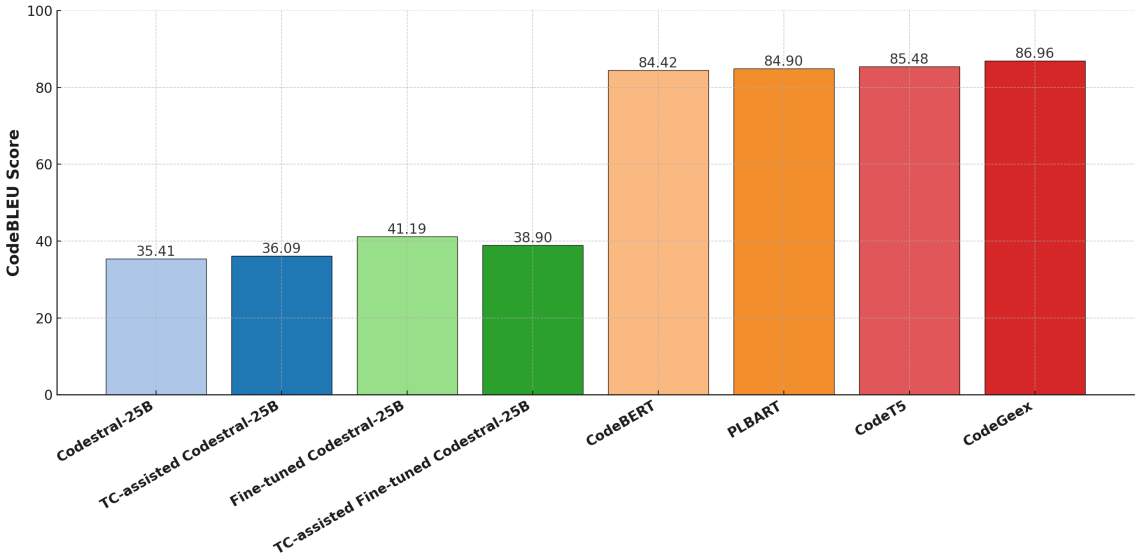

Fig. 6. CodeBLEU scores for JavaScript-to-Java XLCoST dataset across Codestral, fine-tuned Codestral, their TC-assisted versions, and benchmark models

To provide a more meaningful perspective, we applied our execution-based evaluation methodology to the same XLCoST test set. Figure 7 reports the Compilation Rate (CR) and Execution Match (EM) scores for all configurations, including the XLCoST reference translations. Notably, Codestral and TC-assisted Codestral have the highest EM and CR scores. The TC-assisted version has slightly improved the baseline model by 2%, outperforming even its fine-tuned counterpart in functional correctness. The difference between Figure 6 and Figure 7 suggests that CodeBLEU, being a surface-level similarity measure, does not adequately reflect functional correctness. This is very clear from the low CR and EM scores of the XLCoST Reference, which is considered the golden truth or the correct translation from the dataset perspective. Furthermore, the fine-tuned model and its TC-assisted fine-tuned model got lower accuracies than the baseline model and its TC-assisted version due to training on a noisy dataset. The results also indicate that compiler-guided prompting improved translation without additional training, demonstrating its effectiveness in producing executable and behaviorally accurate code. Interestingly, while the TC-assisted prompting slightly reduced the Compilation Success Rate (CR) for some configurations, it maintained or improved the Execution Output Match (EM). This divergence between CR and EM is meaningful: it indicates that the TC-assisted setup discourages the model from producing superficially valid but semantically incorrect programs. In other words, the base model often overfits to produce any code that compiles, regardless of whether it correctly implements the original functionality, whereas the TC-assisted variant generates fewer but more behaviorally accurate translations. The lower CR but stable or higher EM therefore reflects a reduction in compilability bias, where models prioritize syntactic validity over semantic fidelity.

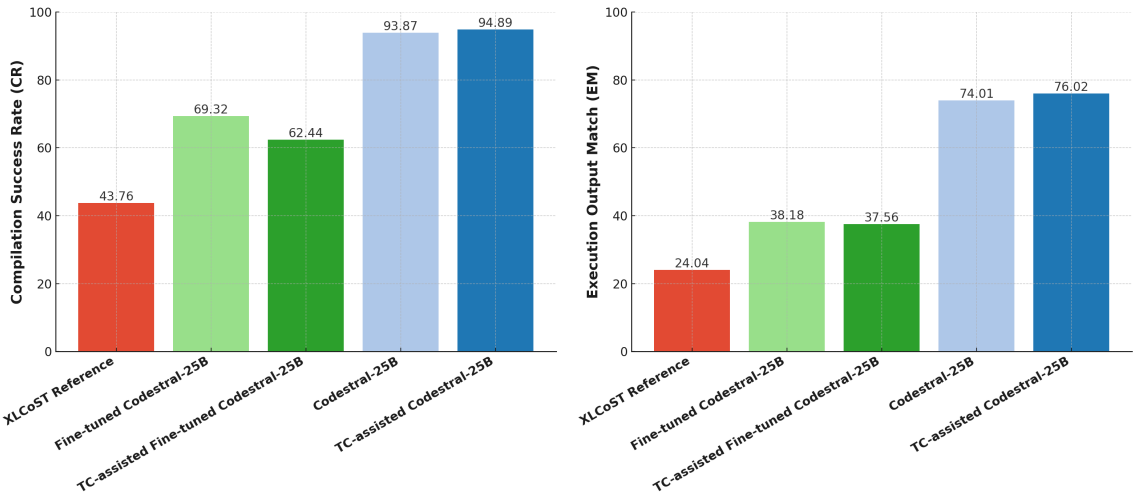

Fig. 7. Compilation Success Rate and Execution Output Match across XLCoST reference, Codestral, fine-tuned Codestral, and TC-assisted versions

Overall, the XLCoST experiments reinforce two key observations. First, CodeBLEU alone can misrepresent model performance when structural similarity diverges from semantic fidelity. Second, the TC-assisted approach enhances code translation by combining compiler-informed structure with LLM flexibility—achieving near-perfect compilation rates and high output match scores, better than model fine-tuning with noisy data.

8 Threats to Validity

This study has some limitations that guide future extensions of the proposed methodology. First, the presented evaluation was conducted using instruction-tuned LLMs (Codestral, GPT-4o-mini, and Qwen-Coder) that are trained to follow natural-language prompts. This choice was made to ensure applicability of our TC-assisted prompting methodology. However, other high-performing code-specific models, particularly those trained exclusively on code corpora but not instruction-tuned, were not included in this study, as they cannot directly interpret NL-style guidance or structured prompts. Future work will explore methods for adapting our TC-assisted framework to these models, potentially by introducing intermediate representations or structured adapters that allow partial transcompiler translations to serve as compatible inputs.

Second, all evaluated models—GPT-4o-mini, Codestral-25B, and Qwen-Coder-32B—were accessed through APIs. While this approach enables reproducible testing, it restricts visibility into model internals such as quantization, decoding parameters, or architectural modifications applied by service providers. In addition, due to this restrictiion. In particular, Qwen-Coder was accessed through a third-party API (OpenRouter), and its exact configuration may differ from the open-source release. Thus, while the improvements from TC-assisted prompting are observed, absolute performance comparisons should be interpreted cautiously. In addition, for the benchmark dataset, XLCoST, we could apply the Codestral model only due to resource restrictions, which also limited our experiments.

Third, the evaluation relies on execution-based metrics that compare the string similarity of program outputs. Although this approach captures functional behavior more accurately than CodeBLEU, it cannot guarantee full semantic equivalence across diverse input scenarios. Achieving stronger correctness guarantees would require generating and executing test cases with multiple input variations and measuring success using metrics such as pass@k, as introduced in prior work by Chen et al. [8]. However, the datasets used in this study do not include paired test cases for each code snippet, limiting such analysis. For deeper verification, integrating formal methods—such as program equivalence checking [24]—would enable systematic proof of correctness under all possible execution conditions.

Despite these constraints, the consistent performance gains across multiple models and translation tasks demonstrate that TC-assisted prompting is model-agnostic and practical for a wide range of instruction-tuned LLMs. Future work could extend this framework to additional languages, integrate richer compiler feedback loops, or combine prompting with lightweight continual learning to further bridge the gap between syntactic validity and verified functional equivalence.

9 Conclusion

This paper presented TC-assisted prompting methodlogy, a hybrid methodology that integrates transcompiler-generated partial translations into the prompting process of instruction-tuned large language models (LLMs). By embedding structured, compiler-informed context, the approach improves the syntactic validity and semantic accuracy of cross-language code translations.

Empirical results for the curated dataset , TGAlpaca, showed consistent gains across all evaluated models and translation directions. Execution output match (EM) scores improved by 2–40%, while on the XLCoST benchmark, the TC-assisted Codestral-25B outperformed the fine-tuned model and the plain Codetsral model. These findings demonstrate that lightweight prompt augmentation can achieve competitive performance relative to specialized fine-tuned systems, while remaining scalable and cost-efficient.

Beyond performance improvements, the proposed methodology also enhances trust in LLM-generated code. By grounding model outputs in verifiable compiler outputs, the approach reduces the frequency of hallucinated constructs, enforces adherence to language syntax, and improves functional correctness. This trust-oriented design is particularly relevant for safety-critical or production-level applications, where deterministic behavior and interpretability are as important as accuracy.

Future work will focus on enhancing the TC-assisted prompting methodology through four main directions. First, improvements in transcompiler quality can further strengthen prompt guidance, yielding more semantically complete and accurate translations while reducing hallucinated code. Second, integrating formal equivalence checking [24] into the translation pipeline would enable a fully verified workflow, ensuring functional correctness between source and translated code. Third, extending this approach to additional programming languages, larger software projects, and adaptive feedback loops, where compiler feedback continually refines model outputs, can expand its applicability and reliability in real-world software engineering contexts. Finally, future work can extend the methodlogy to embed compiler-based partial translation into models that are not instruction tuned to be able to apply and test other category of models that are also code specific and can achieve superior performance.

10 Replication Package

The TGAlpaca dataset curated in this study, all resulting codes from the various setups used, and a sample notebook with the TC-assisting prompting approach are available on this *link*.

11 Acknowledgments

This work was supported in part by the OEAD organization project No. P017-2022. The authors of this paper express their gratitude to Prof. Jens Knoop, head of Research Unit Compilers and Languages at TU Wien, for his valuable review of this article.

References

[1] Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Unified pre-training for program understanding and generation. *arXiv preprint arXiv:2103.06333* (2021).

[2] Toufique Ahmed, Kunal Suresh Pai, Premkumar Devanbu, and Earl T. Barr. 2024. Automatic Semantic Augmentation of Language Model Prompts (for Code Summarization). *Proceedings - International Conference on Software Engineering* (2024), 2720 – 2732. doi:10.1145/3597503.3639183 Cited by: 10; All Open Access, Green Open Access, Hybrid Gold Open Access.

[3] Kijin An, Na Meng, and Eli Tilevich. 2018. Automatic inference of Java-to-swift translation rules for porting mobile applications. *Proceedings of the 5th International Conference on Mobile Software Engineering and Systems - MOBILESoft '18*, 180–190. doi:10.1145/3197231.3197240

[4] Rameez Barakat, Moataz Bellah A. Radwan, Walaa M. Medhat, and Ahmed H. Yousef. 2023. Trans-Compiler-Based Database Code Conversion Model for Native Platforms and Languages. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* 13761 LNAI (2023), 162–175. doi:10.1007/978-3-031-21595-7_12/TABLES/7

[5] Nghi D.Q. Bui and Lingxiao Jiang. 2018. Hierarchical learning of cross-language mappings through distributed vector representations for code. *Proceedings - International Conference on Software Engineering* (2018), 33–36. Issue 2. doi:10.1145/3183399.3183427

[6] Walter Cazzola and Luca Favalli. 2024. Software modernization powered by dynamic language product lines. *Journal of Systems and Software* 218 (2024). doi:10.1016/j.jss.2024.112188 Cited by: 1; All Open Access, Green Open Access, Hybrid Gold Open Access.

[7] Fuxiang Chen, Fatemeh H. Fard, David Lo, and Timofey Bryksin. 2022. On the Transferability of Pre-trained Language Models for Low-Resource Programming Languages. *IEEE International Conference on Program Comprehension* 2022-March (2022), 401 – 412. doi:10.1145/3524610.3527917 Cited by: 31.

[8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[9] Xinyun Chen, Chang Liu, and Dawn Song. [n. d.]. Tree-to-tree Neural Networks for Program Translation.

[10] Zou Donglan, Mohamad Yusof Bin Darus, and Azlin Binti Ramli. 2024. Investigating Developer Experiences with UI Components in Flutter: Challenges and Implications. In *2024 International Visualization, Informatics and Technology Conference (IVIT)*. IEEE, 128–133.

[11] Shaymaa Sayed El-Kaliouby, Sahar Selim, and Ahmed H. Yousef. 2021. Native Mobile Applications UI Code Conversion. *Proceedings - 2021 16th International Conference on Computer Engineering and Systems, ICCES 2021* (2021). doi:10.1109/ICCES54031.2021.9686093

[12] Mikhail Evtikhiev, Egor Bogomolov, Yaroslav Sokolov, and Timofey Bryksin. 2023. Out of the BLEU: How should we assess quality of the Code Generation models? *Journal of Systems and Software* 203 (2023). doi:10.1016/j.jss.2023.111741

[13] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020).

[14] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A pre-trained model for programming and natural languages. *Findings of the Association for Computational Linguistics Findings of ACL: EMNLP 2020* (2020), 1536 – 1547. https://www.scopus.com/inward/record.uri?eid=2-s2.0-85101859312&partnerID=40&md5=df6b4c952b058d00c826170b2211d33b Cited by: 1021.

[15] Clémentine Fourrier, Nathan Habib, Alina Lozovskaya, Konrad Szafer, and Thomas Wolf. 2024. Open LLM Leaderboard v2. https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard.

[16] Rameez B. Hamza, David I. Salama, Martina I. Kamel, and Ahmed H. Yousef. 2019. TCAIOSC: Application Code Conversion. *NILES 2019 - Novel Intelligent and Leading Emerging Sciences Conference*, 230–234. doi:10.1109/NILES.2019.8909207

[17] Mohammed H. Hassan, Omar A. Mahmoud, Omar I. Mohammed, Ammar Y. Baraka, Amira T. Mahmoud, and Ahmed H. Yousef. 2020. Neural Machine Based Mobile Applications Code Translation. *2nd Novel Intelligent and Leading Emerging Sciences Conference, NILES 2020*, 302–307. doi:10.1109/NILES50944.2020.9257935

[18] Prithwish Jana, Piyush Jha, Haoyang Ju, Gautham Kishore, Aryan Mahajan, and Vijay Ganesh. 2024. CoTran: An LLM-Based Code Translator Using Reinforcement Learning with Feedback from Compiler and Symbolic Execution. *Frontiers in Artificial Intelligence and Applications* 392 (2024), 4011 – 4018. doi:10.3233/FAIA240968 Cited by: 3; All Open Access, Hybrid Gold Open Access.

[19] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).

[20] Xuan Li, Shuai Yuan, Xiaodong Gu, Yuting Chen, and Beijun Shen. 2024. Few-shot code translation via task-adapted prompt learning. *Journal of Systems and Software* 212 (2024), 112002. doi:10.1016/j.jss.2024.112002

[21] Zhe Li and Xinxi Lu. 2025. Research on Compressed Input Sequences Based on Compiler Tokenization. *Information (Switzerland)* 16, 2 (2025). doi:10.3390/info16020073 Cited by: 0; All Open Access, Gold Open Access.

[22] Marcos Macedo, Yuan Tian, Pengyu Nie, Filipe R. Cogo, and Bram Adams. 2025. InterTrans: Leveraging Transitive Intermediate Translations to Enhance LLM-Based Code Translation. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering* (Ottawa, Ontario, Canada) *(ICSE '25)*. IEEE Press, 1153–1164. doi:10.1109/ICSE55347.2025.00236

[23] Amira T Mahmoud, Walaa Medhat, Sahar Selim, Hala Zayed, Ahmed H Yousef, and Nahla Elaraby. 2025. TC-Verifier: Trans-Compiler-Based Code Translator Verifier with Model-Checking. *Applied System Innovation* 8, 3 (2025), 60.

[24] Amira T. Mahmoud, Ahmad A. Mohammed, Mahitap Ayman, Walaa Medhat , Sahar Selim , Hala Zayed, Ahmed H. Yousef, and Nahla Elaraby . 2024. Formal Verification of Code Conversion: A Comprehensive Survey. *Technologies* 12, 12 (2024). doi:10.3390/technologies12120244

[25] Amira T. Mahmoud, Ahmad A. Muhammad, Ahmed H. Yousef, Walaa Medhat, Hala H. Zayed, and Sahar Selim. 2023. Compiler-based Web Services code conversion model for different languages of mobile application. *1st International Conference of Intelligent Methods, Systems and Applications, IMSA 2023* (2023), 464–469. doi:10.1109/IMSA58542.2023.10217471

[26] Amira T Mahmoud, Ahmad A Muhammad, Ahmed H Yousef, Hala H Zayed, Walaa Medhat, and Sahar Selim. 2023. Industrial Practitioner Perspective of Mobile Applications Programming Languages and Systems. *I'JACSA) International Journal of Advanced Computer Science and Applications* 14 (2023). Issue 5. www.ijacsa.thesai.org

[27] Amira T Mahmoud, Moatazbellah Radwan, Abdelrahman Mohamed Soliman, Ahmed H Youssef, Hala H Zayed, and Walaa Medhat. 2024. Trans-Compiler-Based Conversion from Cross-Platform Applications to Native Applications. *Annals of Emerging Technologies in Computing (AETiC)* 8, 4 (2024). doi:10.33166/AETiC.2024.04.001

[28] Ahmad A. Muhammad, Amira T. Mahmoud, Shaymaa S. Elkalyouby, Rameez B. Hamza, and Ahmed H. Yousef. 2020. Trans-Compiler based Mobile Applications code converter: swift to java. *2020 2nd Novel Intelligent and Leading Emerging Sciences Conference (NILES)*, 247–252. doi:10.1109/NILES50944.2020.9257928

[29] Ahmad Ahmad Muhammad, Abdelrahman Mohamed Soliman, Sahar Selim, and Ahmed H Yousef. 2021. Generic Library Mapping Approach for Trans-Compilation. In *2021 International Mobile, Intelligent, and Ubiquitous Computing Conference (MIUCC)*. IEEE, 62–68.

[30] Takehiko Nawata and Reiji Suda. 2011. APTCC: Auto Parallelizing Translator From C To CUDA. *Procedia Computer Science* 4 (2011), 352–361. doi:10.1016/j.procs.2011.04.037 Proceedings of the International Conference on Computational Science, ICCS 2011.

[31] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. In *The Eleventh International Conference on Learning Representations*. https://openreview.net/forum?id=iaYcJKpY2B_

[32] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2024. Lost in Translation: A Study of Bugs Introduced by Large Language Models while Translating Code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 82, 13 pages. doi:10.1145/3597503.3639226

[33] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297* (2020).

[34] David I. Salama, Rameez B. Hamza, Martina I. Kamel, Ahmad A. Muhammad, and Ahmed H. Yousef. 2019. TCAIOSC: Trans-Compiler Based Android to iOS Converter. *Advances in Intelligent Systems and Computing* 1058 (10 2019), 842–851. doi:10.1007/978-3-030-31129-2_77

[35] Nourhan M Shaban, Ahmed H Yousef, and Hoda K Mohamed. 2021. General Trans-Compiler based Mobile Applications Converter. In *2021 3rd Novel Intelligent and Leading Emerging Sciences Conference (NILES)*. IEEE, 117–124.

[36] Qiushi Sun, Nuo Chen, Jianing Wang, Xiang Li, and Ming Gao. 2023. Transcoder: Towards unified transferable code representation learning inspired by human skills. *arXiv preprint arXiv:2306.07285* (2023).

[37] Marc Szafraniec, Baptiste Rozière, Hugh Leather, François Charton, Patrick Labatut, and Gabriel Synnaeve. 2023. CODE TRANSLATION WITH COMPILER REPRESENTATIONS. *11th International Conference on Learning Representations, ICLR 2023* (2023). https://www.scopus.com/inward/record.uri?eid=2-s2.0-85199885485&partnerID=40&md5=000d4a273c8debeb600b42df9be00282 Cited by: 14.

[38] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* 2017-December.

[39] Chaozheng Wang, Yuanhang Yang, Cuiyun Gao, Yun Peng, Hongyu Zhang, and Michael R. Lyu. 2022. No more fine-tuning? an experimental evaluation of prompt tuning in code intelligence. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Singapore, Singapore) *(ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 382–394. doi:10.1145/3540250.3549113

[40] Chaozheng Wang, Yuanhang Yang, Cuiyun Gao, Yun Peng, Hongyu Zhang, and Michael R. Lyu. 2023. Prompt Tuning in Code Intelligence: An Experimental Evaluation. *IEEE Trans. Softw. Eng.* 49, 11 (Oct. 2023), 4869–4885. doi:10.1109/TSE.2023.3313881

[41] Deze Wang, Boxing Chen, Shanshan Li, Wei Luo, Shaoliang Peng, Wei Dong, and Xiangke Liao. 2023. One Adapter for All Programming Languages? Adapter Tuning for Code Search and Summarization. *Proceedings - International Conference on Software Engineering* (2023), 5 – 16. doi:10.1109/ICSE48619.2023.00013 Cited by: 31.

[42] Jun Wang, Chenghao Su, Yijie Ou, Yanhui Li, Jialiang Tan, Lin Chen, and Yuming Zhou. 2025. Translating to a Low-Resource Language with Compiler Feedback: A Case Study on Cangjie. *IEEE Transactions on Software Engineering* 51, 9 (2025), 2671 – 2692. doi:10.1109/TSE.2025.3594908 Cited by: 0.

[43] Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. 2021. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *arXiv preprint arXiv:2109.00859* (2021).

[44] Yanzhao Wang and Fei Xie. 2025. Enhancing Translation Validation of Compiler Transformations with Large Language Models. *International Journal of Software Engineering and Knowledge Engineering* 35, 1 (2025), 45 – 57. doi:10.1142/S0218194024500475 Cited by: 0; All Open Access, Green Open Access.

[45] Yuxiang Wei, Federico Cassano, Jiawei Liu, Yifeng Ding, Naman Jain, Zachary Mueller, Harm de Vries, Leandro von Werra, Arjun Guha, and Lingming Zhang. 2024. SelfCodeAlign: Self-Alignment for Code Generation. *Advances in Neural Information Processing Systems* 37 (2024). https://www.scopus.com/inward/record.uri?eid=2-s2.0-105000510231&partnerID=40&md5=9e4a0fe726d24a21e8ed2e0161405630 Cited by: 0.

[46] Yue Wu, Yaoxiang Yu, Zhengming Yuan, Siwei Huang, and Bo Cai. 2024. APT: Adaptive Prefix-Tuning on Pretrained Models for Code Intelligence. *Proceedings of the International Joint Conference on Neural Networks* (2024). doi:10.1109/IJCNN60899.2024.10651243 Cited by: 0.

[47] Weixiang Yan, Yuchen Tian, Yunzhe Li, Qian Chen, and Wen Wang. 2023. CodeTransOcean: A Comprehensive Multilingual Benchmark for Code Translation. *Findings of the Association for Computational Linguistics: EMNLP 2023* (2023), 5067 – 5089. doi:10.18653/v1/2023.findings-emnlp.337 Cited by: 6; All Open Access, Green Open Access, Hybrid Gold Open Access.

[48] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, Teng Su, Zhilin Yang, and Jie Tang. 2023. CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Benchmarking on HumanEval-X. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Long Beach, CA, USA) *(KDD '23)*. Association for Computing Machinery, New York, NY, USA, 5673–5684. doi:10.1145/3580305.3599790

[49] Ming Zhu, Aneesh Jain, Karthik Suresh, Roshan Ravindran, Sindhu Tipirneni, and Chandan K Reddy. 2022. Xlcost: A benchmark dataset for cross-lingual code intelligence. *arXiv preprint arXiv:2206.08474* (2022).

[50] Ming Zhu, Mohimenul Karim, Ismini Lourentzou, and Daphne Yao. 2024. Semi-Supervised Code Translation Overcoming the Scarcity of Parallel Code Data. *Proceedings - 2024 39th ACM/IEEE International Conference on Automated Software Engineering, ASE 2024* (2024), 1545 – 1556. doi:10.1145/3691620.3695524 Cited by: 0.

[51] Ming Zhu, Karthik Suresh, and Chandan K. Reddy. 2022. Multilingual Code Snippets Training for Program Translation. *Proceedings of the 36th AAAI Conference on Artificial Intelligence, AAAI 2022* 36 (2022), 11783 – 11790. doi:10.1609/aaai.v36i10.21434 Cited by: 22; All Open Access, Gold Open Access.

Received November 2025; revised ; accepted